

Nous allons, au travers de divers exemples simples, nous sensibiliser au JavaScript.

Il faut travailler progressivement et tester votre page web à chaque étape (ex regardez la page, regarde les outils de développement, vérifier votre HTML avec).

Rappel: Règles de base du langage

- Chaque ligne d'instructions se termine par un ;
- Le début d'un bloc d'instructions est précédé du symbole { et la fin du bloc d'instruction est suivie du symbole }
- Une ligne de commentaires doit commencer par /* et se terminer par */ ou commence par //
- Le code JavaScript s'écrit, dans un fichier HTML, entre les balises <script></script> (y compris quand le code appelle un script dans un autre fichier)
- Le JavaScript est sensible à la casse, donc attention aux majuscules et minuscules.

1. Les bases

1.1. Premiers pas avec JavaScript

Q1 :

Créer une page HTML avec le code suivant :

```
<!DOCTYPE html>
<html lang="fr">

  <head>
    <meta charset="utf-8">
    <title>Test</title>
  </head>

  <body>

    <h1>Un titre</h1>
    <p>Du texte...</p>

    <script>
      alert("Nous interrompons cette page Web pour une petite annonce...");
    </script>

  </body>
</html>
```

Q2 :

L'idée est de créer une fonction pour appeler cet dialogue d'alerte. Essayez d'abord sans regarder le code suivant (qui le fait effectivement).

```
<!DOCTYPE html>
<html lang="fr">

  <head>
    <meta charset="utf-8">
    <title>Test</title>
```

```
<script>
    function affichageMsg (s) {
        alert("Nous interrompons cette page Web pour une petite
annonce..." + s);
    }
</script>
</head>

<body>

    <h1>Un titre</h1>
    <p>Du texte...</p>

    <script>
        affichageMsg("Hello");
    </script>

</body>
</html>
```

À noter que le paramètre "s" de la fonction `affichageMsg` n'a pas un type (comme en C/C++).

Pour voir l'effet d'ordre, essayez de mettre la déclaration de la fonction affichage après son appel - cela va donner l'erreur `Uncaught ReferenceError: affichageMsg is not defined` qui nous rappelle que l'ordre avec lequel on introduit les scripts est importante.

Q3 :

Il est possible d'externaliser le code JavaScript, un peu à la manière du code CSS, en indiquant dans la page HTML, quelque chose comme :

```
<script src="fichier.js"></script>
```

Il est d'usage d'insérer le lien vers le fichier JavaScript dans l'entête du code HTML.

Créer un fichier nommé `code.js` contenant le code de la fonction `affichageMsg()`. Modifier le fichier HTML de la question précédente pour utiliser le fichier `code.js` (Attention : le fichier HTML ne doit plus contenir le code de la fonction...). Vérifier toujours le bon fonctionnement de la page.

Q4 :

Essayez de jouer un peu avec le langage JavaScript, par exemple ajoutez une boucle avec `for` dans la fonction et mettez des msg dans la console avec la fonction `console.log("un msg")`.

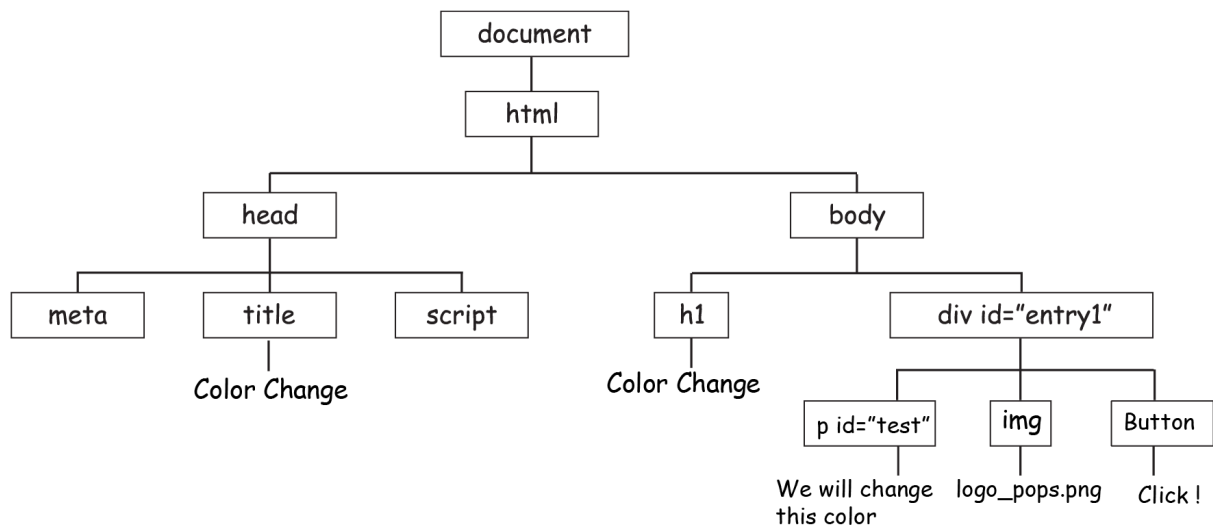
Par exemple comptez jusqu'à 10 et imprimer chaque numéro, puis créez un tableau avec quelques chiffres ou mots et imprimer chaque élément du tableau.

1.2. Modification du DOM (Document Object Model) d'une page

Qu'est-ce que le DOM ?

Pour faire simple, le DOM représente l'arborescence des différents éléments présents sur un document HTML. Il définit également une manière d'accéder et d'agir sur chacun de ces éléments. Le JavaScript permet d'agir sur le DOM via une API : pour modifier, ajouter ou supprimer une balise.

Voici un DOM :



Q5:

Essayez de créer l'HTML que ce DOM décrit. Essayez d'abord seul.e.s (ne regardez pas encore l'étape 7 car il y a dedans des elements de la solution).

Notre objectif finale sera d'agir sur le DOM à l'aide du JavaScript.

Q6:

La fonction ou méthode du `document` `getElementById ( )` permet de trouver une balise HTML en fonction de son `id`. Il est ensuite possible d'agir dessus (son contenu, son style, etc.).

Nous allons alors ajouter un événement `onclick` dans le bouton. Quand le bouton est cliqué nous allons trouver l'élément avec `id "test"`. Et en utilisant la paramètre `innerHTML` nous pouvons changer le texte de l'élément "test" par exemple avec le text "color has been changed". On est optimiste car on a pas encore changé de couleur, mais cela arrive !

Un exemple très similaire existe dans les notes du cours. Essayez d'abord sans regarder la solution (qui est dans l'étape 7). Si les choses ne fonctionnent pas, testez votre code pour des erreurs (inspectez votre page et regardez les msg dans vos utils de développement).

Q7 :

Saisir le code suivant (solution de Q5 et Q6) et le tester.

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="utf-8">
    <title>Color Change</title>
  </head>
  <body>
    <h1>Color Change </h1>

    <div id="entry1">
      <p id="test"> We will change this color ...</p>

      <button
        onclick="document.getElementById('test').innerHTML=
          'changed color';"> Click!
      </button>
    </div>
  </body>
</html>
```

Remarque : `onclick` est ce que l'on appelle un événement. Quand cet événement survient, le navigateur exécute le code JavaScript donné après le `=`. Il existe de nombreux autres événements comme `onmouseover` (quand le curseur de la souris est positionné sur l'élément HTML), `onmouseout` (quand le curseur quitte un élément), `onkeydown` (lors de l'appui sur une touche), `onload` (quand le navigateur a fini de charger la page HTML), `onmousedown` (quand l'utilisateur appuie sur un bouton de la souris), etc.

Q8 :

Afin de rendre le code plus lisible remplacer le code après le `=` de `onclick` par l'appel à une fonction nommée `gestionclik()`. Par exemple :

```
<button onclick="gestionclik();">Click!</button>
```

Modifier le fichier HTML afin d'obtenir le fonctionnement attendu.

Q9 :

Il est également possible de modifier les attributs d'une balise HTML autre que `innerHTML`, par exemple `href`, `src`, etc. Modifier le fichier précédent de manière à faire apparaître une image affichant par défaut l'image `logo_pops.png`. Ajouter 2 boutons et le texte comme montré ci-dessous.

Logo Polytech Paris-Saclay

En rouge ou en bleu...



L'appui sur le bouton Rouge doit faire afficher l'image `logo_rouge.png` à la place de l'image de départ (src de votre img), et le bouton Bleu doit afficher l'image de départ.

1.3. Modification du style

Q10 :

Le JavaScript autorise également la modification du style d'éléments HTML, tout comme le ferait une page CSS. La syntaxe sera la suivante :

```
document.getElementById("test").style.proprietedestyle="nouvellevaleur";
```

pour `proprietedestyle`, il faudra remplacer par le nom de la propriété CSS en un seul mot :

- si la propriété est à l'origine écrite en un mot, e.g., `margin`, `weight`, `color`, etc. le nom reste identique en JavaScript ;
- si la propriété est à l'origine écrite en avec un tiret -, e.g., `text-align`, `border-color`, etc., en JavaScript le tiret est supprimé et la lettre suivante devient une majuscule : `textAlign`, `borderColor`.

Souvent, il est nécessaire d'appliquer plusieurs changements de style à la même balise, à ce moment, il est souvent préféré d'utiliser la syntaxe suivante :

```
let i = document.getElementById("test");  
i.style.textAlign="center";  
i.style.borderWidth="3px";
```

Modifier la page de la question précédente pour faire en sorte que le titre `Logo Polytech Paris-Saclay` passe également en rouge et en italique lors de l'appui sur le bouton Rouge, et en bleu et un style normal lors de l'appui sur le bouton Bleu.

Q11 :

Nous décidons de supprimer les boutons et d'effectuer les changements du titre et de l'image lorsque la souris est sur le mot rouge ou le mot bleu. De plus le mot passera en gras en fonction de la couleur active. Modifier le code précédent pour obtenir le fonctionnement désiré.

Logo Polytech Paris-Saclay

En rouge ou en bleu...



Remarque : Il existe deux balises HTML dont le seul rôle est de délimiter une zone associée à un style : `<div></div>` pour les blocs de texte (à la manière d'un paragraphe), `<span></span>` pour un ou plusieurs mots dans un texte.

Par exemple :

```
<div id="texte1">
  <p>Du texte pour remplir et surtout donner un bon exemple
    de l'utilisation de ces balises.</p>
  <p>Cependant <span id="texte2">attention</span> à ne pas confondre
    ces deux balises!</p>
</div>
```

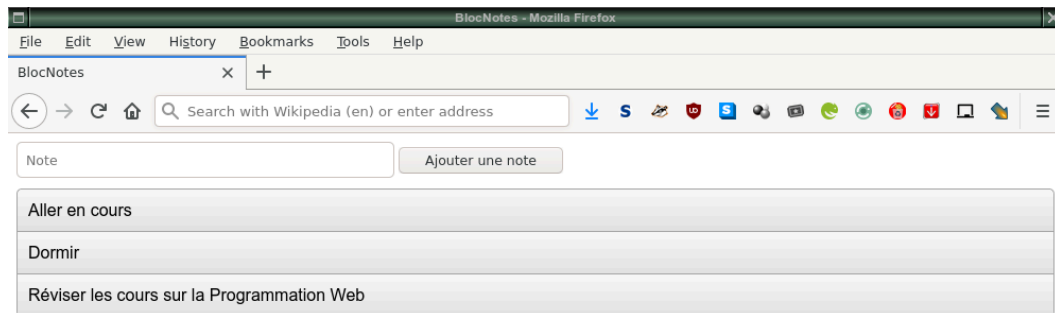
Sans style, ces balises ne produisent rien de particulier.

2. Un exemple plus complexe - Bloc-notes

Un des intérêts du couple JavaScript-HTML5 est la possibilité de créer des applications utilisables sur n'importe quelle plate-forme disposant d'un navigateur compatible HTML5.

Dans ce qui suit, nous allons créer une application de bloc-notes dans laquelle nous pourrions saisir des notes, les effacer, et les conserver sur le disque dur.

Notre application ressemblera à ce qui suit :



Q12 :

Créer le fichier HTML nommé `blocnotes1.html` et contenant le code suivant :

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="utf-8">
    <title>BlocNotes</title>
    <script src="blocnotes1.js"></script>
    <link rel="stylesheet" href="blocnotes.css">
  </head>
  <body>
    <script>
      window.onload=init;
    </script>

    <form>
      <input type="text" id="entreenote" size="40" placeholder="Note">
      <input type="button" id="ajouter" value="Ajouter une note"
onclick="gestionClicBouton();">
    </form>

    <ul id="listenotes">
    </ul>
  </body>
</html>
```

La ligne `window.onload=init;` indique qu'une fois le chargement de la page HTML achevé, la fonction `init()` sera exécutée. Cette fonction est présente dans le script `blocnotes1.js`.

Attention : Il est bien écrit `window.onload=init;` ce qui signifie de l'on affecte à la méthode `onload` de la fenêtre du navigateur le code de la fonction `init()`, qui ne sera donc exécutée qu'une fois la fenêtre complètement chargée. Utiliser la ligne `window.onload=init();` provoquerait l'exécution immédiate de la fonction `init()` lorsque le navigateur lira cette ligne.

En revanche, il est possible d'utiliser à la place l'écriture suivante dans la balise `<body>` :

```
<body onload="init();">
```

où l'on indique la commande à exécuter quand l'événement `onload` de `<body>` se produit, ce qui correspond à `window.onload=init;`.

Autre exemple, à la place d'écrire :

```
<input type="button" id="ajouter" value="Ajouter une note"
onclick="gestionClicBouton();">
```

souvent, nous trouverons (avec `let` ou `var`):

```
<input type="button" id="ajouter" value="Ajouter une note">
<script>
  let b = document.getElementById("ajouter");
  b.onclick = gestionClicBouton;
</script>
```

Là aussi, on associe à la méthode `onclick` le code de la fonction `gestionClicBouton`, on ne souhaite pas exécuter directement la fonction `gestionClicBouton` quand le navigateur lit cette ligne, mais uniquement lors d'un clic souris.

Cette écriture est une des subtilités du JavaScript.

Nous remarquons dans le code HTML, qu'une liste à puces est créée mais sans élément. Les éléments seront créés à l'ajout de chaque note.

Le fichier `blocnote.css` est fourni. Le fichier `blocnotes1.js` est également fourni. Quelques explications du code :

```
/* blocnotes1.js */

function init() {
  console.log("lancement");
}

function gestionClicBouton() {
  let textInput = document.getElementById("entreenote");
  let note = textInput.value;
```



```

    if (note == "") {
        alert("Veuillez entrer une note");
    } else {
        console.log("Ajout de " + note);
        let li = document.createElement("li");
        li.innerHTML = note;
        let ul = document.getElementById("listenotes");
        ul.appendChild(li);
    }
}

```

La fonction `init()`, ne fait qu'afficher une information sur la console du navigateur ou console Web. C'est l'endroit où le navigateur écrit des informations (erreurs, etc.) sur l'affichage des pages Web ou l'exécution des scripts.

Afficher cette console à l'aide de soit avec `inspect > Console` de votre page, soit `Ctrl-Shift-k` et vérifier qu'au chargement de la page le mot lancement est bien affiché.

La fonction `gestionClicBouton()` pour chaque appui sur le bouton `Ajouter`, crée un nouvel élément `<li></li>` contenant le contenu de la note et rajoute cet élément `<li></li>` à l'élément parent `<ul></ul>`.

Il est possible de vérifier que les éléments sont ajoutés en examinant le code HTML créé à l'aide de `inspect > Elements` ou `Ctrl- Shift-c`.

Q13:

Notre bloc-notes permet d'ajouter simplement des notes. Il est nécessaire d'ajouter la possibilité de supprimer une note en cliquant dessus.

Copier `blocnotes1.html` en `blocnotes2.html` et `blocnotes1.js` en `blocnotes2.js`.

Ajouter le code suivant à `blocnotes2.js` :

```

function gestionClicNote(event) {
    let note = event.target;
    let ul = document.getElementById("listenotes");
    ul.removeChild(note);
}

```

Cette fonction va permettre de supprimer l'élément `<li></li>` sur lequel nous avons cliqué (`event.target`).

L'ajout de cette fonction est, pour l'instant, insuffisant pour provoquer la suppression d'une note. Il faut qu'à chaque clic sur une des notes la fonction `gestionClicNote` soit appelée.

Modifier la fonction `gestionClicBouton` pour que chaque élément `<li>` créé se voit attribuer une fonction à exécuter lors d'un clic sur l'élément.

Q14 :

Notre bloc-notes fonctionne, mais si l'on recharge la page toutes les notes sont perdues.

Avec l'HTML5 de nombreuses possibilités sont apparues comme le stockage Web local ou stockage DOM (ou Web storage ou DOM storage) qui permet d'enregistrer des données, de manière persistante, dans le navigateur ou plus précisément dans un fichier géré par le navigateur (Cf. http://www.w3schools.com/html/html5_webstorage.asp).

Le navigateur Firefox et Chrome stocke les données de stockage Web local dans un fichier nommé `webappsstore.sqlite`. Ce fichier n'est pas directement lisible, c'est une base de données de type SQL. On l'appelle souvent local storage (sauvegarde locale, sauvegarde coté client) et on l'utilise pour les données non-sensibles (ex. un panier).

Le fichier `blocnotes_stockage.js` fourni des fonctions permettant le stockage et la récupération des notes de notre bloc-notes, c-à-d des fonctions pour accéder local storage. On utilise le format JSON (dans le fichier il y a plusieurs références pour ce format).

Copier `blocnotes2.html` vers `blocnotes3.html` et `blocnotes2.js` vers `blocnotes3.js`.
Rajouter à `blocnotes3.html` l'appel au fichier `blocnotes_stockage.js`.

Q15 :

Nous voulons :

- qu'à chaque ajout de note, celle-ci soit sauvegardée ;
- qu'à chaque chargement de la page `blocnotes3.html`, les notes soient également chargées et affichées ;
- qu'à chaque clic sur une note, la note en plus d'être supprimée de l'écran soit également supprimée de la sauvegarde.

Modifier le fichier `blocnotes3.js` à cet effet en utilisant les fonctions du fichier `blocnotes_stockage.js`. Il sera possible de contrôler le contenu du stockage local en faisant `Shift-F9` puis `Local Storage` OU `inspect > Application > Storage > Local Storage`.

Q16 :

Copier `blocnotes3.html` vers `blocnotes4.html` et `blocnotes3.js` vers `blocnotes4.js`.

En s'aidant de http://www.w3schools.com/jsref/met_win_confirm.asp ajouter une confirmation avant de supprimer une note.

Q17 :

Copier `blocnotes4.html` vers `blocnotes5.html` et `blocnotes4.js` vers `blocnotes5.js` ainsi que `blocnotes_stockage.js` vers `blocnotes_stockage5.js`.

Actuellement, il est possible d'ajouter plusieurs notes avec le même contenu, ce comportement n'est, en fait, pas désiré. Rajouter une vérification de l'absence d'une note identique avant son ajout et afficher un message si l'utilisateur tente d'ajouter une note identique à une autre. Pensez à utiliser `IndexOf` qui renvoie -1 si un élément n'est pas dans un tableau.

3. JavaScript et canvas

Avec l'HTML 5.0 est apparu le *canvas*. C'est un nouvel élément (`<canvas>`) qui définit une zone graphique sur la page HTML dans laquelle on peut dessiner en 2D ou en 3D. Pour y accéder, il faut utiliser le JavaScript. De base, il est possible de réaliser des tracés de courbes, de l'affichage de textes et d'images mais là où cela devient intéressant est qu'avec un peu de code JavaScript, il est possible d'y faire des animations, etc.

Pour l'utiliser, il faut un navigateur récent supportant l'HTML 5.0 (c'est normalement le cas de tous les navigateurs aujourd'hui).

Pour plus d'information sur le canvas: http://www.w3schools.com/html/html5_canvas.asp et https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API.

3.1. Exemple de base d'utilisation du canvas

Q18 :

Le fichier `ex-canvas.html` donne un exemple basique de l'utilisation du canvas. Ouvrez cette page avec Firefox, puis examiner le code du fichier.

3.2. Gestion du clavier en JavaScript et canvas

Q19 :

Modifier l'exemple précédant de manière à déplacer au clavier le carré vert (de 5 en 5) à l'aide des touches k (haut), j (bas), l (droite) et h (gauche). Il faudra veiller à ce que le carré ne puisse pas sortir du canvas.

4. Mise en œuvre

Nous allons, au travers de deux exemples appliquer vos connaissances du JavaScript.

4.1. Calculatrice

On vous donne les fichiers de base `calculatrice.html` et `style.css`. L'objectif est de créer une calculatrice qui :

- Propose un calcul à droite de la zone de saisie du résultat (une multiplication de deux nombres aléatoires entre 1 et 10), par exemple $5 \times 3 =$;
- En cliquant sur **Vérifier** (ou en appuyant sur **Entrée**) vérifie si le calcul est bon, félicite le participant dans une fenêtre alert si la réponse est bonne, ou lui donne la bonne réponse sinon. Un nouveau calcul est proposé sans recharger la page ;
- Le score est calculé en direct: on affiche le nombre de bonnes réponses sur le nombre de réponses total. Ce score est de couleur rouge si l'utilisateur a moins de la moyenne, et vert sinon.

Pour la génération d'un nombre aléatoire, on pourra utiliser la fonction JavaScript suivante:

```
function hasard(n) {
    return Math.ceil(Math.random()*n);
}
```

L'événement qui nous intéresse est `onkeydown` et vous pouvez récupérer le touche de l'événement `e` avec `e.keyCode`;

4.2. Menus déroulants

Les fichiers de base `menu.html` et `menuVertical.css` sont donnés. L'objectif est d'avoir des menus qui se cachent ou se montrent quand on clique sur un menu.

Les menus 1 et 3 n'ont pas de sous-menu: cliquer dessus doit tout fermer.

Pour les menus 2 et 4, si on clique alors que leur sous-menu est fermé, on ferme tout le reste et ouvre le leur. Si leur sous-menu est ouvert, on le ferme lors d'un clic dessus.

Considérez l'utilisation du css style `display` (avec la valeur `none` ou `block`).

Quoi soumettre :

A la fin de l'exercice, créez un zip qui contient :

- vos fichiers `.html`
- vos fichiers `.css`
- vos fichiers `.js`
- vos répertoire, par exemple `images/`

Téléchargez-le sous `Activity-3-JS` dans `ecampus`.